

Table of Contents

Week 7 — Summary & Examples Reference

Alternate Real-World Examples for Every Part of the Lesson

One-Line Definitions (Read These Aloud)

Part 1 Alternate Examples — What Is Method Overriding?

Alternate Story — Insurance Policy Local Amendments

Alternate Story — Hospital Department Greetings

Alternate Story — Employee Report Format

Part 2 Alternate Examples — The Override Declaration: Saying It Clearly

Alternate Story — Legal Contract Amendment

Alternate Story — Building Permit Modification

Part 3 Alternate Examples — Extend vs Replace: Parent's Behavior First

Alternate Analogy — Mobile App Updates

Alternate Analogy — Hospital Standard Operating Procedure

Alternate — Sealed Behavior Story

Part 4 Alternate Examples — Overriding vs Overloading

Alternate Story — Hotel Check-in Desk (Overloading)

Alternate Story — Pharmacy Dispense System (Overriding)

Part 5 Alternate Example — Java as Illustration

Concept Translation Table (Employee Version)

Overriding vs Overloading — Practice Sets

Set A — Quick Classification

Set B — Students Discuss

Student Discussion Questions

Common Student Confusions — Quick Fixes

Lesson Arc Summary (Mapped to Both Example Sets)

Homework Reminder

Week 7 — Summary & Examples Reference

Alternate Real-World Examples for Every Part of the Lesson

Use alongside: Week7_Method_Overriding.md | **Class:** Wednesday, 15 July 2026 · 6:30 PM

*This file mirrors the main lesson guide part-by-part, but every example here is **different** from the main file. Use these when a student needs a second angle, or to add variety mid-discussion without repeating yourself.*

One-Line Definitions (Read These Aloud)

Method Overriding is when a child class provides its own version of an action the parent already defined — same name, same inputs, but the child's behavior replaces the parent's for that type.

Extend strategy means the child runs the parent's version first, then adds its own content on top.

Replace strategy means the child ignores the parent's version entirely and starts fresh.

Sealed behavior means an action is locked — no child class is permitted to change it.

Overloading is different — same action name in the same class, but each version has different inputs.

Part 1 Alternate Examples — What Is Method Overriding?

Main file used: KFC franchise + university introduce() + vehicle fuel cost.

This file uses: insurance policy local amendments + hospital department greetings + employee report format.

Alternate Story — Insurance Policy Local Amendments

"An insurance company's headquarters defines the standard claims process: submit a claim, provide documentation, wait 14 days, receive payment. Every regional office starts with this process."

But the northern rural region added a modification: because many clients there have no internet, claims can also be submitted by paper form at the local post office. The northern region's claims process looks slightly different from headquarters' version — it runs differently for that specific region.

The northern region has **overridden** the standard claims process with its own local version."

INSURANCE CLAIMS OVERRIDE:

Parent (headquarters): processClaim() → online form, 14 days
Child (northern region): processClaim() → online form OR paper form
at post office, 14 days

Why override?

The northern region IS-A regional office (same type of thing) but its specific process is different from the default.

Alternate Story — Hospital Department Greetings

"A hospital has a general patient greeting script: 'Welcome to City Hospital. How can we help you today?' Every department starts with this.

But the Emergency Department uses: 'Welcome to City Hospital Emergency — please tell us your emergency immediately.' The Children's Ward uses: 'Welcome to City Hospital! 🌟 What's your little one's name?'

Both departments override the standard greeting with one specific to their patients' needs."

HOSPITAL DEPARTMENT GREETINGS – THREE VERSIONS:

Parent (hospital):	<code>greetPatient()</code> → standard welcome script
Child (Emergency Dept):	<code>greetPatient()</code> → urgency-focused greeting
Child (Children's Ward):	<code>greetPatient()</code> → child-friendly greeting

All three are hospital departments (IS-A relationship). Each overrides the greeting action with its own version.

Alternate Story — Employee Report Format

WHICH VERSION RUNS – APPLIED TO REPORTS:

Company policy: all employees submit weekly reports.

Employee type: SoftwareEngineer

When HR requests a weekly report from "this employee" –
is it a generic employee report or an engineer's report?

The actual person IS a SoftwareEngineer.

Even if HR just thinks of them as "an Employee" –
when the report comes in, it follows the ENGINEER's format.

The actual type of the person determines which report format
is used – not how HR labelled them in the system.

This is exactly how overriding works in OOP:
the actual object type determines which version runs.

Part 2 Alternate Examples — The Override Declaration: Saying It Clearly

Main file used: written employment contract for modified reporting + typo story (Introduce vs introduce).

This file uses: a legal contract amendment and a construction building permit.

Alternate Story — Legal Contract Amendment

"A business partnership agreement has standard terms both parties signed. Later, one party wants to change the payment schedule from monthly to quarterly. This change must be:

- 1. Written down explicitly — not just verbally agreed*
- 2. Signed by both parties to confirm it is intentional*
- 3. Clearly attached to the original contract*

Without this explicit, formal amendment, the original monthly payment schedule remains valid. If someone just starts paying quarterly without a formal amendment, there could be a dispute: 'Did we agree to this? When? Where is the record?'"

LEGAL AMENDMENT → OOP OVERRIDE DECLARATION:

Written explicitly → declaring the override intent clearly
Both parties confirm → the system verifies the parent has
that action and the override is valid
Attached to original → the child's version is connected to
and replaces the parent's version

Without formal amendment → the original version still applies
Without override declaration → typos silently create new actions
instead of replacing the parent's

Alternate Story — Building Permit Modification

"A construction company has a standard floor plan approved by the city. If they want to modify the design — add an extra window, change a wall — they must formally submit a 'design modification notice' and get it approved. They cannot just quietly build something different and hope no one notices.

Similarly, when a child class wants to replace a parent's action, it should formally declare this intent — so the system can verify it is valid, not a mistake."

SILENT MODIFICATION PROBLEM (no declaration):

The construction company quietly builds an extra door.
The city plans still show the original design.
Inspector checks plans against the building → mismatch found
→ The hidden change causes problems when discovered later

WITH FORMAL DECLARATION:

Company submits modification notice.
City checks: "Is this change compatible with the foundation?"
If yes → approved and recorded. If no → flagged immediately.
→ Problems caught before the building is done, not after.

Part 3 Alternate Examples — Extend vs Replace: Parent's Behavior First

Main file used: national bank branch + local welcome greeting (government regulation story) + sealed food safety checklist + national exam format.

This file uses: mobile app version updates and hospital standard procedure.

Alternate Analogy — Mobile App Updates

"When a new version of an app is released, it doesn't throw away everything the previous version built. It keeps the core features (the parent) and ADDS improvements or changes specific behavior where needed (the child overrides). The 'Settings' screen still exists — it just has new options added.

Only when something is fundamentally broken in the old version do developers replace it entirely."

APP UPDATE STRATEGIES – EXTEND VS REPLACE:

Feature: `showProfilePage()`

EXTEND strategy (most updates):

Old version displays: name, photo, bio

New version displays: name, photo, bio (same as before)
+ new: badges, follower count

→ The old content is still there; new content added on top

REPLACE strategy (redesign):

Old version: classic list layout

New version: card-based grid layout
(everything about the display is different)

→ The old layout is completely gone; entirely new design

Alternate Analogy — Hospital Standard Operating Procedure

"A hospital has a standard procedure for patient discharge: check vitals, print discharge letter, schedule follow-up. The Cardiology department follows this procedure AND adds: an ECG printout and a dietary restriction sheet. They EXTEND the standard.

The ICU department, however, has a completely different discharge process — patients need intensive documentation, specialist sign-offs, and ambulance coordination. They REPLACE the standard entirely. The standard procedure doesn't apply to ICU discharges."

HOSPITAL DISCHARGE PROCEDURES:

Parent (standard): `discharge()` → vitals, letter, follow-up

Cardiology (EXTEND): `discharge()` → runs parent's steps FIRST
 then adds: ECG, diet sheet

ICU (REPLACE): `discharge()` → completely different process
 parent steps don't apply here

Which strategy?

Ask: "Is the parent's version still valid for this child?"

If YES → EXTEND (keep parent, add own)

If NO → REPLACE (start fresh)

Alternate — Sealed Behavior Story

"A pharmaceutical company has a standard drug safety verification procedure. This procedure must be followed identically across ALL product types — no division or team is allowed to modify it. The CEO has declared it 'non-negotiable.'

In OOP terms: the safety verification action is sealed. A child class (any product type) that tries to override it receives an immediate error — the system enforces the non-negotiable rule automatically."

Part 4 Alternate Examples — Overriding vs Overloading

Main file used: English word "call" + KFC franchise vs café multiple `order()` versions.

This file uses: *a hotel check-in desk and a pharmacy dispense system.*

Alternate Story — Hotel Check-in Desk (Overloading)

"A hotel receptionist can check in a guest in multiple ways:

- `checkIn(guestName)` — quick walk-in, no reservation*
- `checkIn(guestName, reservationCode)` — standard booking*
- `checkIn(guestName, reservationCode, specialRequest)` — booking with a request*

All three are called 'check in.' The receptionist decides which process to use based on WHAT the guest provides at the desk. This decision is made the moment the guest arrives — not before.

*This is **overloading** — same name, different inputs, decision made at the moment of use."*

Alternate Story — Pharmacy Dispense System (Overriding)

"A pharmacy chain has a standard `dispenseProduct()` process. But specialty pharmacies (compounding pharmacies) that mix custom medications override this process — their `dispenseProduct()` involves additional weighing, mixing, and quality checks.

*A compounding pharmacy IS-A pharmacy. Its `dispenseProduct()` replaces the standard one. This decision is made based on WHICH TYPE of pharmacy it is — not on what inputs you give. This is **overriding**."*

SIDE-BY-SIDE COMPARISON:

HOTEL CHECK-IN (OVERLOADING)	PHARMACY DISPENSE (OVERRIDING)
Same name: <code>checkIn()</code>	Same name: <code>dispenseProduct()</code>
Different inputs decide which version runs	Different object TYPE decides which version runs
Decision at: moment of call	Decision at: moment of call
Same class	Requires inheritance (child type)
No inheritance needed	CompoundingPharmacy IS-A Pharmacy

Part 5 Alternate Example — Java as Illustration

Main file showed *Person* → *Student* / *Teacher* with `introduce()` and `getRole()`.

This file shows the same concept using *Employee* → *SoftwareEngineer* / *Designer* with `submitReport()` and `getRole()`.

```
// PARENT - defines default report format for all employees
public class Employee {
    protected String name;
    protected String department;

    public Employee(String name, String department) {
        this.name = name;
        this.department = department;
    }

    // Default report - generic for all employees
    public void submitReport() {
        System.out.println("[REPORT] " + name + " - " + department
            + " | Weekly summary submitted.");
    }
}
```

```

        public String getRole() {            return "Employee";    }    }
// CHILD - engineer overrides report format (EXTEND strategy)
public class SoftwareEngineer extends Employee {    private String currentProject;
    public SoftwareEngineer(String name, String dept, String project) {
        super(name, dept);            this.currentProject = project;    }                                @Override
    public void submitReport() {
        super.submitReport();            // run parent's version first (EXTEND)
        System.out.println(" Project: " + currentProject
            + " | Code commits: tracked in GitHub");    }
    @Override    public String getRole() {
        return "Software Engineer"; // REPLACE - completely different answer    }    }
    // ANOTHER CHILD - designer also overrides (EXTEND strategy)
public class Designer extends Employee {    private String designTool;
    public Designer(String name, String dept, String tool) {            super(name, dept);
        this.designTool = tool;    }                                @Override    public void submitReport() {
        super.submitReport();            // EXTEND - keep parent content
        System.out.println(" Tool used: " + designTool
            + " | Mockups delivered: tracked in Figma");    }
    @Override    public String getRole() {            return "Designer";    }                                // REPLACE
    }    }

```

```

Employee[] team = {
    new SoftwareEngineer("Sokha", "Engineering", "OOP Platform"),
    new Designer("Dara", "Design", "Figma"),
    new Employee("Chan", "Admin")
};

for (Employee e : team) {
    e.submitReport();
    System.out.println("Role: " + e.getRole());
    System.out.println();
}

```

OUTPUT:

```

[REPORT] Sokha - Engineering | Weekly summary submitted.    ← from parent
  Project: OOP Platform | Code commits: tracked in GitHub    ← engineer adds
Role: Software Engineer

[REPORT] Dara - Design | Weekly summary submitted.          ← from parent
  Tool used: Figma | Mockups delivered: tracked in Figma    ← designer adds
Role: Designer

[REPORT] Chan - Admin | Weekly summary submitted.           ← no override → parent runs
Role: Employee

```

Concept Translation Table (Employee Version)

Java code	OOP concept meaning
<code>@Override</code>	"Declaring: I intentionally replace the parent's version"
<code>super.submitReport()</code>	"Run the parent's version first (EXTEND strategy)"
<code>getRole()</code> returns "Designer"	"Fully replaced – no parent content kept"
<code>eng.submitReport()</code>	"Engineer's version runs – declared intent confirmed"
<code>e.getRole()</code> in loop	"Actual object type determines which version runs"

Overriding vs Overloading — Practice Sets

Set A — Quick Classification

Scenario	Answer	Why
<code>Employee.getRole()</code> and <code>SoftwareEngineer.getRole()</code> (same inputs, child)	Overriding	Same name, same inputs, child replaces parent
A <code>Calculator</code> class has <code>add(int, int)</code> and <code>add(double, double)</code>	Overloading	Same name, different input types, same class
<code>Hospital.greetPatient()</code> and <code>EmergencyDept.greetPatient()</code> (same inputs, child)	Overriding	Child type replaces parent behavior
<code>Order</code> class has <code>process()</code> , <code>process(String note)</code> , <code>process(String note, boolean rush)</code>	Overloading	Same name, different inputs, same class
<code>Person.introduce()</code> and <code>Student.introduce(String language)</code> (different input)	Neither	Different inputs → not an override; different class → not a standard overload

Set B — Students Discuss

Scenario	Expected Answer
<code>BankAccount.withdraw(amount)</code> and <code>SavingsAccount.withdraw(amount)</code>	Overriding — child type has different rules for the same action
<code>Printer.print(text)</code> and <code>Printer.print(text, copies)</code> in the same class	Overloading — same class, different inputs
<code>Pharmacy.dispense()</code> and <code>CompoundingPharmacy.dispense()</code>	Overriding — child has a different process
<code>format()</code> , <code>format(String type)</code> , <code>format(String type, int width)</code> in same class	Overloading — three versions, different inputs

Student Discussion Questions

1. "Give a real-world example from your own life of something that was 'inherited' by default (a standard procedure or policy) but that your version of the situation adapted or replaced. Was it an EXTEND or REPLACE strategy?"
2. "In the legal contract amendment story — what happens if you change something without a formal amendment? How does that connect to what happens when you override without declaring intent?"
3. "Why might a pharmaceutical company want to SEAL its safety verification procedure? Can you think of something in your own university or workplace that should be sealed and never changed by any team?"
4. "Look at the hotel check-in (overloading) example. Why is this NOT inheritance? What is the key difference that makes it overloading instead of overriding?"
5. "The loop at the end of the Java illustration stores Engineers, Designers, and plain Employees all in the same array. When `submitReport()` is called, each runs its own version. Why? What decides which version runs?"

Common Student Confusions — Quick Fixes

If a student says...	Respond with...
"Override and overload are the same thing"	"Hotel check-in uses overloading — same receptionist, different information from the guest. Pharmacy uses overriding — different TYPE of pharmacy, different process. Type vs inputs — completely different triggers."
"I need to call the parent's version first every time"	"Not always. Ask: 'Is the parent's version still useful for my child?' ICU doesn't follow standard discharge — it replaces entirely. Cardiology adds to it — extends. Your choice."
"If I write @Override, it makes the override happen"	"No — @Override is a declaration and a check. Like a formal contract amendment: it's a record that you INTEND to replace, and it asks the system to verify the replacement is valid."
"Overriding changes the parent class"	"Like a branch adapting a global procedure — headquarters is unaffected. Person's introduce() is exactly the same after Student overrides it."

Lesson Arc Summary (Mapped to Both Example Sets)

HOW THE TWO EXAMPLE SETS LINE UP, PART BY PART

PART	MAIN FILE EXAMPLE	THIS FILE'S EXAMPLE
Part 1 – What Is Overriding?	KFC franchise + University introduce() + Vehicle fuel cost	Insurance policy amendments + Hospital greetings + Employee report format
Part 2 – Declaring Intent	Employment contract + Introduce typo story	Legal contract amendment + Building permit modification
Part 3 – Extend vs	Bank branch regulation +	Mobile app updates +

exam format

procedures +

Pharma sealed safety

Part 4 – Override vs
Overloading

English "call" word +
KFC vs café order()

Hotel check-in desk
(overloading) +
Pharmacy dispense
(overriding)

Part 5 – Java
Illustration

Person → Student,
Teacher
(introduce, getRole)

Employee →
SoftwareEngineer,
Designer
(submitReport, getRole)

Same concepts. Different stories. Use whichever lands best.

Homework Reminder

Optional Java code — plain English design is sufficient.

Design Person → Student + Teacher with overriding:

- State for each override: EXTEND or REPLACE strategy, and why
- Write expected output for 2 students and 2 teachers from a shared Person list

Week 7 Summary Reference | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com